

ACORN 蓝图

Yifeng Xiao

August 19, 2026

CONTENTS

ACORN 蓝图	
一、文献定位：别人控制什么，我们控制什么	
二、骨架：两种状态，一个回路	
Verifiable self-evolving：合约库的进化回路（离线，与运行时同核）	
三、运行时输出设计：decision frame	
四、评测指标：把「优势」变成可测量的数字	
合规（双轴）	
交互效率	
控制经济学（ACORN 特有）	
鲁棒性	
库与进化（verifiable）	

ACORN 蓝图

Agent Contract Orchestration for Runtime Navigation —— 一个 neuro-symbolic agent harness 的架构、文献定位、运行时输出设计与评测指标体系。

程序性合约被编译成**逐步的可容许动作空间**：还有真选择时收窄给 LLM 选（dynamic tool exposure）；程序已唯一确定下一步时，直接执行、不调模型（symbolic jump-forward）。

三个核心贡献点：① dynamic action-space compilation ② symbolic action execution ③ neural-symbolic control handoff；外加 verifiable self-evolving contract library。

一、文献定位：别人控制什么，我们控制什么

相关工作可以按「在 agent 决策回路的哪个点、什么时机、用什么机制施加控制」分成五族。关键区分维度是：**veto**（事后/临门否决）还是 **shape**（事前重塑动作空间），以及是否会代替模型执行。

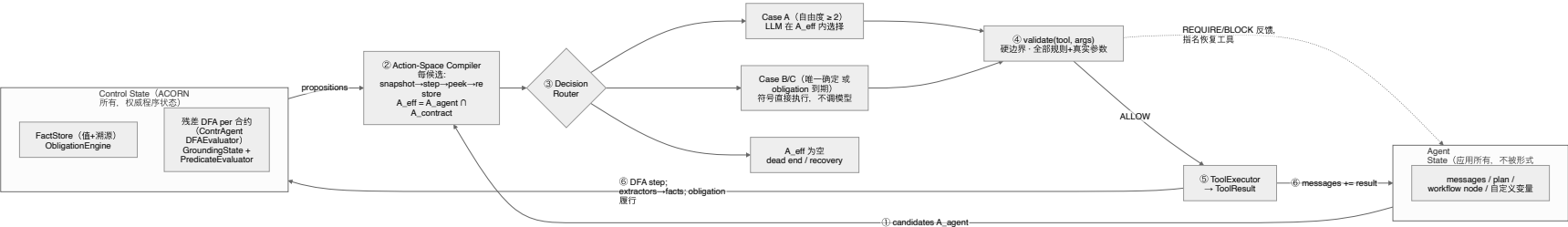
族	代表工作	控制点与时机	与 ACORN 的差异
编排 框架	LangGraph、OpenAI Agents SDK、AutoGen、CrewAI、smolagents、Claude Agent SDK	工作流图 / 回调钩子；程序逻辑写进 prompt 或 graph	把 SOP 编码成 workflow 造成状态积（20 节点 × 30 合约态）；guardrail 只是钩子回调。ACORN 让合约独立于编排存在，运行时求交： $A_{eff} = A_{agent} \cap A_{contract}$
运行时 执法	Plug in the Safety Chip (ICRA'24)、AgentSpec、Progent、ShieldAgent、GuardAgent、 ContrAgent	动作边界，pre-execution veto ：提案 → 检查 → 拦截 + 重提示	这是我们的「passive verifier」基线（ContrAgent 在 SOPBench 上 86/82）。全族都是 check-only：不重塑动作空间、不代执行、无 obligation 语义。被拦提案的 token 与恢复轮次全是浪费
对话 轨约 束	NeMo Guardrails (Colang)、Guardrails AI、Llama Guard	对话流 / 输出 schema / 内容分类	控制的是「说什么」，不是「做什么」；无时序工具合约，无世界状态 grounding
形式 化约 束生 成	Formal-LLM（自动机约束规划）、SayCan（affordance 掩码）、LTL 约束 RL；token 级：Outlines / XGrammar / guidance / SGLang、Monitor-Guided Decoding	约束 生成过程 ：plan 级自动机或 token 级 FSM 掩码	最接近「shape 而非 veto」的一族，但要么离线 plan 级、要么只管语法合法性。没有证据/义务状态，不是通用 tool-calling harness。token 级掩码是我们 deferred 的 优化后端 ，语义真源仍在 controller
自进 化 agent	Voyager（技能库）、ADAS / AFlow（自动化 workflow 搜索）、self-evolving agents survey (2025)	进化 prompt / 代码 / 工作流，靠任务奖励验证	ACORN 进化的对象是 合约库 ：从轨迹/SOP 语料归纳（ContrAgent evolve/，12 个 empirical patterns，82.7% 覆盖率）→ 修复算子 → LTLf-SAT 子随检查去冗 → 冲突自由证书。进化产物是 可形式验证的 ，而非只有 reward 信号

三个组合起来没有先例的点：**(1)** 时序合约在每一步被编译成状态相关的可容许动作集（而不是提案后检查）；**(2)** 程序唯一确定下一步时符号直接执行（省掉模型调用，obligation 不可被「结束会话」逃逸）；**(3)** 控制权在神经与符号之间按步动态移交，

evidence / obligation 是一等运行时状态。

二、骨架：两种状态，一个回路

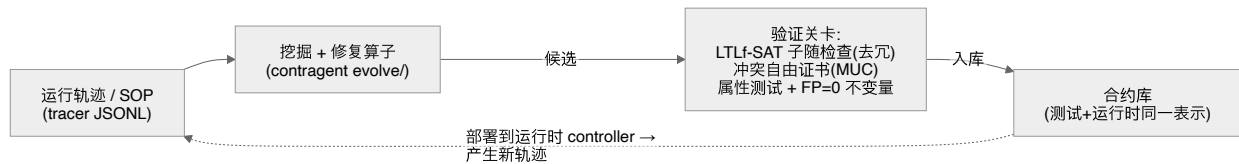
Agent State 归应用所有（ACORN 从不形式化它）；Control State 归 ACORN 所有（权威的程序性状态）。两者只在回路里交互，从不合并成积自动机。符号内核（grounding + 残差 DFA + LTLf-SAT）全部复用 ContrAgent。



ACORN 运行时回路

②的探针成本是 $O(|\text{候选}| \times |\text{合约}|)$ 、每次 $O(1)$ （残差 DFA snapshot/restore），这是能把 veto 升级成 shape 的工程前提。

Verifiable self-evolving：合约库的进化回路（离线，与运行时同核）



合约库进化回路

与「reward 驱动的 prompt/ workflow 进化」的差别在验证关卡：每条入库合约都带形式化证书（不与库内合约冲突、不被已有合约蕴含、在允许任务上零误报），进化是可审计的。

三、运行时输出设计：decision frame

现有 harness 的输出各有侧重，但都只记录**发生了什么**：LangGraph/LangSmith 流式输出节点事件与状态增量的 span 树；OpenAI Agents SDK 输出 RunItems (message / tool_call / handoff / guardrail tripwire)；Claude Code 流式展示 tool_use 与权限确认；smolagents 打印 thought→code→observation 步日志；Inspect AI 面向评测输出 transcript + scorer。

没有一家能回答反事实问题：「为什么这个工具这一步不可用？」「这一步是谁决定的？」这正是 ACORN 输出的差异点。我们的原子单位是 **decision frame** —— 每步一帧，三条设计原则：

- **每个缺席都有署名理由**：被 mask 的工具逐个标注触发的合约（可审计的反事实）；
- **每一步都有权威标签**：neural / symbolic / dead-end，谁做的决定一目了然；
- **每个事实都有溯源**：fact 的建立（哪个工具的结果）、失效（哪条 INVALIDATES）、触发了哪条 obligation。

已实现的渲染（examples/bank_demo.py 真实输出节选，欺诈路径）：

```

— step 2 ————— bank_demo · fraud path
  [masked] replace_card      ← requires identity_verified,
  fraud_checked, address_confirmed
  [neural] exposed: lookup_customer verify_identity check_fraud
  confirm_address ...
  model proposes → check_fraud({"customer_id": "C-1024"})
  executed check_fraud ok=True
    + fact fraud_checked = True          source=check_fraud
    + fact fraud_detected = True
    ! OBLIGATION: fraud detected → freeze the account immediately

— step 3 —————
  [SYMBOLIC] freeze_account({"customer_id": "C-1024"}) (obligation)
  [no model call]
    * obligation satisfied

— step 4 —————
  [masked] replace_card      ← forbidden while fraud_detected
  [neural] exposed: 7 tools
  model proposes → replace_card({"customer_id": "C-1024"})
  ** BLOCK ** replace_card: forbidden while fraud_detected – do not
  retry

metrics: model_calls=5 · symbolic_steps=2 · blocked=1 ·
symbolic_ratio=0.40

```

底层是结构化 JSONL 事件流：`controller/decision`、`action/masked`、`action/symbolic`、`action/blocked`、`fact/asserted|invalidated`、`obligation/created|satisfied|pending`、`model/response`、`tool/result`、`contract/violation|final`。评测报告直接由同一 JSONL 聚合——测试与运行时同一数据面。v0 的 JSONL + 终端渲染已经够用；逐帧回放 CLI 与 HTML viewer 是后续可选项，研究版本不做。

四、评测指标：把「优势」变成可测量的数字

对比条件固定四档：**prompt-only**（SOP 进提示词）→ **passive verifier**（ContrAgent 式拦截+重提示，我们的强基线）→ **+ dynamic exposure** → **+ jump-forward**。每一档只加一个机制，指标的增量即机制的贡献。

合规（双轴）

指标	定义
task success	SOP 允许的任务中目标完成率
safety	SOP 禁止的任务中目标未完成率
committed violation rate	已执行动作里违约的比例（enforce 下应恒为 0，量基线用）

prompt-only 用 success 换 safety（SOPBench 上 85→62）；passive verifier 双高（86/82）。ACORN 需证明 mask+jump 不损失 success 的同时把 committed violation 压到构造性为 0。

交互效率

指标	定义
invalid-proposal rate	被拦 / 未知工具提案 ÷ 全部提案
recovery turns	一次拦截到合规重试的轮次数
wasted tokens	无效提案消耗的输入+输出 token

veto 系统只能事后拦，无效提案与恢复轮次是其固有开销；mask 让模型根本看不到不可用工具，该组指标应随条件档位单调下降。

控制经济学（ACORN 特有）

指标	定义
Symbolic Execution Ratio	符号执行的程序步 ÷ 全部程序步
Neural Decision Ratio	需要 LLM 决策的步 ÷ 全部动作选择步
freedom profile	逐步 IA_effl / IA_agentl 的分布
obligation latency	trigger→discharge 的步数（ACORN 确定性 = 1）
schema token savings	动态暴露相对全量工具 schema 的 token 差

其他 harness 结构上没有这些量：没有 A_eff 就没有自由度剖面；没有 obligation 就没有响应延迟可言（prompted 的 freeze 可能晚 k 轮甚至不发生，ACORN 恒为下一步）。schema token 节省在 Amazon SOP-Bench 的大工具注册表上最显著。

鲁棒性

指标	定义
weak-model retention	safety(弱模型) ÷ safety(强模型)
adversarial compliance	用户施压/改述下 safety 保持率
dead-end rate	IA_effl=0 且无恢复路径的会话占比

ContrAgent 已证实：决定论对模型不敏感（flash-lite 上 prompt 崩、enforce 稳）。加上 mask 后预期弱模型的 success 也被抬起（选择题变简单）。dead-end rate 检验 mask 不会把 agent 逼进死路。

库与进化 (verifiable)

指标	定义
coverage	SOP 门被合约覆盖的比例 (可审计的谓词→事实映射集)
conflict-freedom	库级 SAT 证书 (含 witness-trace 快速路径)
FP rate	允许任务上被错误 mask/block 的比例 (目标恒 0)

自进化系统的通病是「进化了但没验证」。我们每轮进化交付三张证书：冲突自由 (LTLf-SAT/MUC)、无冗余 (子随检查)、允许任务零误报 —— 这组指标本身就是论文的 verifiable 论据。

基准选择：Amazon SOP-Bench 为主报告目标 (工业 SOP + 大工具注册表, tool-selection 与 schema-token 指标最能拉开差距)；学术 SOPBench (7 客服域) 作为 parity 锚点 —— passive verifier 条件须先复现 ContrAgent 的 86/82, 证明重构无回归, 再叠加两个机制读增量。

acorn v0 : 12/12 tests · bank_demo 两场景可复现 (fraud path: model_calls=5, symbolic_ratio=0.40) · 符号内核复用 ContrAgent (formulas / grounding / DFAEvaluator / sat)